

Углубленное машинное обучение и искусственный интеллект в Python

Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com



Что такое временные ряды

Временной ряд — это последовательность наблюдений x_t , индексированных временем t . Данные собираются в равномерные или неравномерные интервалы времени.



Ключевое свойство: Зависимость между наблюдениями во времени

$$x_t = f(x_{t-1}, x_{t-2}, \dots, \varepsilon_t)$$

Равномерное

Фиксированный интервал

Неравномерное

Переменный интервал



Примеры данных



Финансы
Цены акций



Медицина
ЭКГ, сигналы



IoT
Датчики



Ритейл
Спрос

⚠ Почему это важно

Время порождает зависимость и причинность. Корректная работа с порядком во времени — ключ к точному прогнозу и избежанию утечек данных.

Примеры задач на временных рядах

Различные типы задач, которые решаются с помощью анализа временных рядов



Прогнозирование

Forecasting

- Точечное прогнозирование
- Вероятностное прогнозирование
- Многошаговое прогнозирование

→ Применение: финансы, погода, спрос



Импутация

Imputation

- Заполнение пропусков
- Сглаживание шума
- Фильтрация данных

→ Применение: медицина, IoT



Детекция аномалий

Anomaly Detection

- Обнаружение отклонений
- Кластеризация событий
- Классификация аномалий

→ Применение: безопасность, фрод



Классификация и сегментация

Classification & Segmentation

- Классификация последовательностей
- Сегментация временных рядов
- Распознавание паттернов
- Анализ ЭКГ, вибраций

→ Применение: медицина, промышленность



Контроль и оптимизация

Control & Optimization

- Управление процессами
- Оптимизация ресурсов
- MLOps для TS
- Автоматизация

→ Применение: IoT, производство

Чем временные ряды отличаются от табличных данных

Ключевые различия между независимыми данными и временными зависимостями



Ключевые отличия

Основные характеристики



Порядок и зависимость во времени

Временные ряды имеют строгий порядок, табличные данные — нет



Автокорреляция

Зависимость от предыдущих значений



Сезонность и тренд

Паттерны, повторяющиеся во времени



Нестационарность

Изменение статистических свойств



Визуальное сравнение

IID-данные vs Временные ряды

● IID-данные



Перемешивание допустимо

● Временные ряды



Строгое разделение по времени



Важно понимать

Временные ряды требуют специального подхода к разделению данных и валидации

Типичные проблемы временных рядов

Ключевые проблемы, с которыми сталкиваются при анализе временных рядов



Автокорреляция

Зависимость от предыдущих значений

Корреляция между наблюдениями на разных временных лагах

$$\rho(k) = \text{Cov}(x_t, x_{t-k}) / (\sigma_t \sigma_{t-k})$$



Нестационарность

Изменение статистических свойств

Среднее, дисперсия или автокорреляция меняются со временем

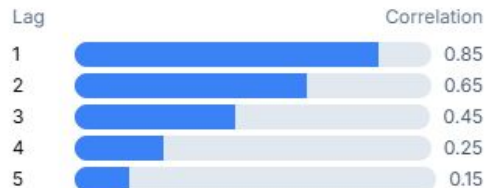
↑ Требуется преобразований



ACF/PACF анализ

Автокорреляционная функция

k=20



Значимые корреляции выделены синим



ACF анализ

Помогает определить порядок AR модели



Последствия

Влияние на анализ

- Смещение оценок
- Переобучение
- Неверные интервалы доверия

0.85

Макс. корреляция

Решение: Преобразования, дифференцирование
Используйте ADF тест для проверки стационарности



Классические методы: постановка и мотивация

Классические методы — это статистические подходы к анализу временных рядов, основанные на линейных стохастических процессах, сезонности и разностях.



Зачем изучать: Служат сильными базовыми линиями, интерпретируемы, работают при малых данных

$$y_t = f(y_{t-1}, \dots, y_{t-p}, \varepsilon_t, \dots, \varepsilon_{t-q})$$

Преимущества

Интерпретируемость, простота

Ограничения

Линейность, ограниченная память



Ключевые концепции

1

Стационарность

Инвариантность распределения по времени

2

Автокорреляция

Зависимость от предыдущих значений

3

Сезонность

Периодические паттерны



Почему это важно

Классические методы — фундамент для понимания современных подходов. Они помогают выявить базовые паттерны и служат бенчмарками для сравнения.

Ключевые недостатки традиционных подходов к анализу временных рядов



Линейность и слабая гибкость

Классические методы предполагают линейные зависимости, что ограничивает их способность моделировать сложные нелинейные паттерны в данных.

- ❗ Не подходит для нелинейных систем



Чувствительность к нестационарности

Требование стационарности делает модели уязвимыми к трендам и сезонным колебаниям.

- ❗ Требует предварительной обработки



Ограниченная дальняя память

Модели неэффективно учитывают дальние зависимости в данных.

- ✅ Короткая история влияния



Чувствительность к выбросам

Классические методы уязвимы к аномалиям и выбросам в данных.

- ⚠ Требуется очистки данных



ML для временных рядов

Машинное обучение позволяет моделировать нелинейные зависимости в данных, использовать богатые признаки и внешние факторы для прогнозирования.



Ключевая идея: Преобразовать задачу в supervised learning

$$Z_t = [x_{\{t-L+1:t\}}, \text{exog}_t] \rightarrow y_{\{t+h\}}$$

Supervised

Обучение с учителем

Feature-rich

Богатые признаки



Подход к ML

- **Feature engineering:** лаги, скользящие статистики, Fourier
- **Supervised learning:** окна признаков → цель
- **Gradient boosting:** XGBoost, LightGBM для TS

Lags

Rolling stats

Fourier

XGBoost

⚠ Почему это важно

ML позволяет моделировать нелинейности, использовать богатые признаки и внешние факторы для более точного прогнозирования.

Feature engineering: лаги и скользящие статистики

Создание признаков для моделей машинного обучения на временных рядах



Лаговые признаки (Lag Features)

$x_{\{t-k\}}$ — значения из прошлого

Lag 1 (предыдущее значение)

$x_{\{t-1\}} = [y_{\{t-1\}}, y_{\{t-2\}}, \dots]$

Lag 7 (недельный)

$x_{\{t-7\}} = [y_{\{t-7\}}, y_{\{t-14\}}, \dots]$

- ✓ Простая интерпретация
- ✓ Учет автокорреляции
- ✓ Легко реализовать



Скользящие статистики

Среднее, стандартное отклонение, мин/макс

Rolling Mean (окно 7)

$mean_t = (1/7) \sum y_{\{t-i\}}$

Rolling Std (окно 7)

$std_t = \sqrt{(\sum (y_{\{t-i\}} - mean)^2 / 7)}$

- ✓ Сглаживание шума
- ✓ Выявление трендов
- ✓ Обнаружение аномалий



Календарные признаки

- day_of_week: 0-6
- hour: 0-23
- month: 1-12
- is_weekend: 0/1



Fourier признаки

- $\sin(2\pi t/T)$
- $\cos(2\pi t/T)$
- Гармоники сезонности
- Циклические паттерны



Отношения лагов

- $y_t / y_{\{t-1\}}$
- $(y_t - y_{\{t-1\}}) / y_{\{t-1\}}$
- Процент изменения
- Логарифмические

Градиентный бустинг для TS и утечки данных

Применение XGBoost/LightGBM для временных рядов и предотвращение data leakage



Gradient Boosting для TS

XGBoost/LightGBM

Преимущества для TS

- Деревья на признаках (лаги, скользящие, Fourier)
- Регуляризация (L1, L2, early stopping)
- Важности признаков

Ключевые параметры

max_depth: 3-6

learning_rate: 0.01-0.1

n_estimators: 100-500

subsample: 0.8-0.9



Data Leakage Warning

Критические ошибки



Не используйте будущие значения

При построении признаков нельзя использовать данные из будущего



Масштабирование только по train

Fit на train, apply на val/test

Корректный CV: Time series cross-validation c rolling window

Классика vs ML: когда и что выбирать

Сравнение ARIMA и Gradient Boosting по ключевым критериям выбора модели

Критерий	ARIMA / Классические методы	Gradient Boosting / ML
Предпосылки Линейность vs Гибкость	 Линейные процессы Стационарность, линейные зависимости  Линейные данные	 Нелинейные процессы Сложные паттерны, взаимодействия  Нелинейные данные
Нелинейности Способность моделировать	Ограниченная способность 30% Требует преобразований	Высокая способность 90% Естественная поддержка
Мультивариативность Многомерные данные	Ограниченная поддержка  VAR модели, но сложно масштабировать	Естественная поддержка  Легко добавлять признаки
Интерпретируемость Понимание модели	Высокая  Понятные коэффициенты	Средняя  Feature importance, но менее прозрачна
Требования к данным Объем и качество	Мало данных  Работает с 50+ наблюдениями	Больше данных  Требует 1000+ наблюдений



Рекуррентные сети: зачем и когда

RNN (Recurrent Neural Networks) — нейронные сети, которые обрабатывают последовательности данных, сохраняя информацию о предыдущих шагах через скрытое состояние h_t .



Ключевая идея: Учет динамики напрямую, поддержка мультивариативных входов и нелинейностей

$$h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$$

Преимущество

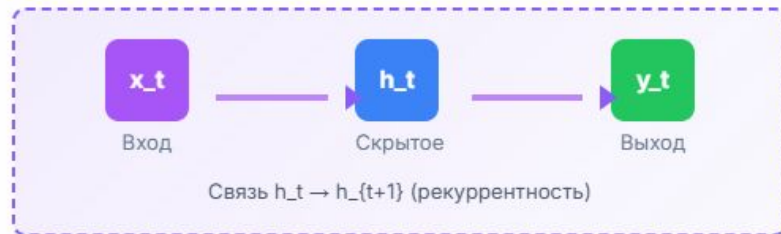
Учет контекста

Применение

Последовательности



Архитектура RNN



Временные
зависимости



Память
о прошлом

⚠ Почему это важно

RNN учат динамику напрямую, поддерживают мультивариативные входы и нелинейности — идеальны для последовательных данных.

Ограничения RNN для длинных последовательностей

Ключевые проблемы, которые ограничивают эффективность рекуррентных сетей



Последовательные вычисления

Sequential Computation

- Нет параллелизма
- Медленное обучение
- Зависимость от предыдущих шагов

→ Сложность: $O(n)$ на шаг



Ограниченная память

Limited Memory

- Забывание дальних зависимостей
- Vanishing gradient
- Короткая память

→ Эффективность: ~100 шагов



Высокие затраты

Computational Cost

- Рост затрат с длиной
- Медленный инференс
- Требования к памяти

→ Затраты: $O(n^2)$ на GPU

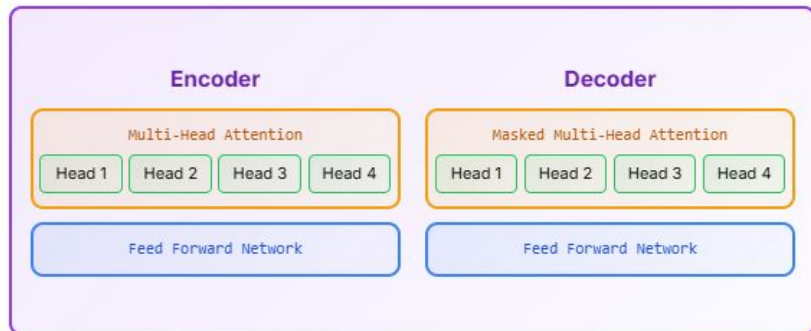
Multi-Head и Transformer encoder-decoder

Архитектура Transformer с multi-head attention для временных рядов



Transformer Architecture

Encoder-Decoder с Multi-Head Attention



Multi-Head Attention: Параллельное вычисление нескольких attention-функций

$Q, K, V \rightarrow \text{Linear} \rightarrow \text{Scaled Dot-Product} \rightarrow \text{Concat} \rightarrow \text{Linear}$



Механизм работы

Пошаговое объяснение



Ключевая идея

Multi-head attention позволяет модели одновременно фокусироваться на разных аспектах данных

- 1 **QKV Projection:** Линейное преобразование входных данных
- 2 **Scaled Dot-Product:** $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$
- 3 **Concat & Linear:** Объединение голов и финальное преобразование

Encoder
6 слоёв, 8 голов

Decoder
6 слоёв, 8 голов

Почему Transformers лучше RNN для длинных зависимостей

Ключевые преимущества архитектуры Transformer над рекуррентными сетями



Параллелизм

Parallel Processing

- ✓ Все позиции обрабатываются одновременно
 - ✓ $O(1)$ временная сложность на GPU
 - ✓ Эффективное использование GPU/TPU
- vs RNN: последовательная обработка



Дальние зависимости

Long-range Dependencies

- ✓ Прямой путь градиента $O(1)$
 - ✓ Нет затухания градиента
 - ✓ Глобальный контекст
- vs RNN: $O(n)$ шагов



Attention

Mechanism

- ✓ Динамическое взвешивание
 - ✓ Адаптивное внимание
 - ✓ Интерпретируемость
- vs RNN: фиксированное состояние

Техническое сравнение

RNN vs Transformer

Временная сложность

$O(n^2)$ vs $O(n)$

Transformer быстрее

Память

$O(n)$ vs $O(n^2)$

RNN эффективнее

Параллелизм

Низкий vs Высокий

Transformer лучше

Дальние зависимости

Слабые vs Сильные

Transformer лучше

Архитектурное сравнение

Схема

RNN

Transformer



i Transformer обрабатывает все позиции параллельно

Transformers для временных рядов

Применение архитектуры Transformer к прогнозированию временных рядов



Архитектура Transformer

Компоненты для временных рядов



Self-Attention механизм

Взвешенная агрегация по всем позициям



Multi-Head Attention

Несколько голов внимания параллельно



Positional Encoding

Информация о позиции во времени



Применение к TS

Ключевые аспекты



Токенизация сигналов

Разбиение ряда на патчи или точки



Masked Attention

Маскирование будущих значений



Multi-step прогноз

Прогнозирование нескольких шагов



Преимущество

Эффективное моделирование длинных зависимостей



Специализированные архитектуры

Почему это важно: Адаптируют attention к длинным рядам, явно моделируют тренд/сезон, улучшают эффективность прогнозирования



Ключевые преимущества:

- Учет долгосрочных зависимостей
- Декомпозиция компонент
- Масштабируемость
- Интерпретируемость



Обзор моделей

Informer

ProbSparse Attention для длинных рядов

$O(L \log L)$

Efficient

Autoformer

Декомпозиция тренд + сезон

Seasonal

Trend

FEDformer

Частотная область attention

FFT

Frequency



Дополнительные модели

PatchTST

Patching для временных рядов

Patching

CNN-like

TFT

Temporal Fusion Transformer

Multi-horizon

Interpretable

Выбор модели

Зависит от длины ряда, сезонности, требований к интерпретации

Autoformer: декомпозиция тренд + сезон

Эксплицитное разложение временного ряда на компоненты с автокорреляционным механизмом



Декомпозиция ряда

$$x_t = \text{trend}_t + \text{seasonal}_t$$

Trend Component

$$\text{trend}_t = \text{MovingAvg}(x_t)$$

70%

Seasonal Component

$$\text{seasonal}_t = x_t - \text{trend}_t$$

30%



Auto-Correlation Mechanism

Заменяет dot-product attention на автокорреляцию, учитывающую периодичность



Архитектура Autoformer

Encoder-Decoder с декомпозицией

- 1 **Series Decomp Block:** Разложение на тренд и сезонность
- 2 **Auto-Correlation:** Внимание на основе автокорреляции
- 3 **Feed Forward:** Позиционно-независимая обработка

$O(L \log L)$

Сложность

Long-term

Память

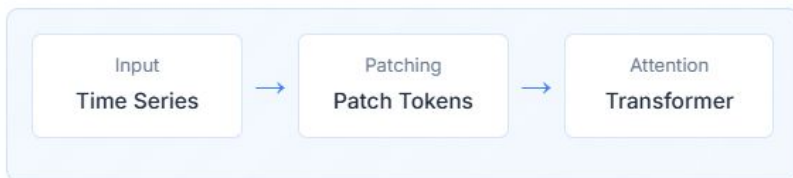
PatchTST и TFT

Современные архитектуры для анализа временных рядов



PatchTST

Patching Time Series



Patching Strategy

Разбиение ряда на перекрывающиеся патчи



CNN-like Tokenization

Линейная проекция в токены



Patch-wise Attention

Эффективное внимание на уровне патчей



Эффективность: $O(L)$ сложность



TFT

Temporal Fusion Transformer



Variable Selection Networks

Автоматический выбор признаков



Temporal Processing

Обработка статических и динамических ковариат



Interpretability

Высокая интерпретируемость модели



Мультивариативность: поддержка



Foundation Models

Foundation models — это большие модели, предобученные на широком корпусе временных рядов для переноса на новые домены.



Ключевая идея: Pretraining на масштабных данных

- Zero-shot прогнозирование
- Few-shot адаптация
- Transfer learning



Почему это важно

Преобучение на широком корпусе рядов → перенос на новые домены (zero/few-shot), экономия данных и разметки



TimesFM

Google

Pretraining

Masked



Chronos

Amazon

Probabilistic

Zero-shot



Moirai

Salesforce

Universal

Multivariate



Lag-Llama

Open Source

Decoder

LLM-based



TimeGPT

Nixtla

Production

API



TinyTime

Lightweight

Efficient

Fast

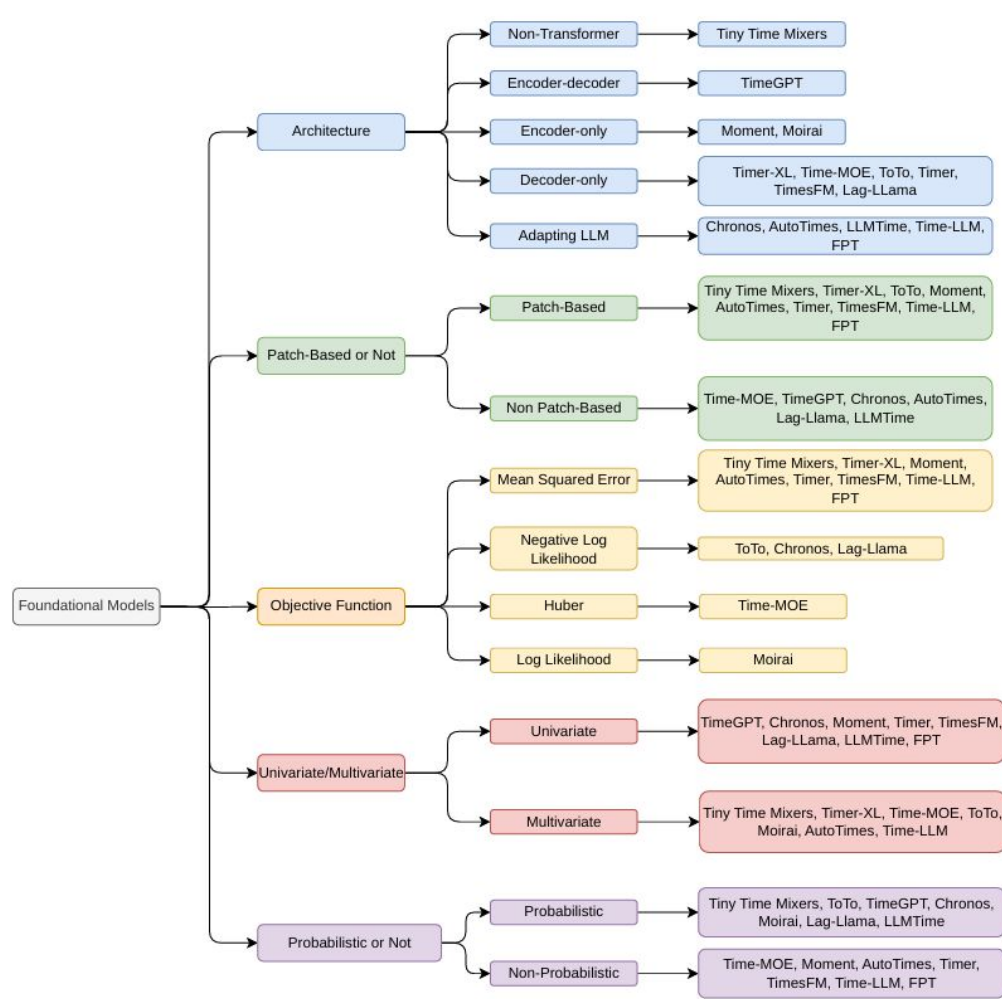
TimesFM, Chronos, Moirai: обзор и сравнение

Сравнение трех ведущих foundation models для временных рядов

Характеристика	TimesFM (Google)	Chronos (Amazon)	Moirai
Разработчик Компания	 Google Research 2024	 Amazon Science 2024	 Salesforce AI 2024
Архитектура Базовая модель	Decoder-only Transformer Masked forecasting 	Probabilistic Transformer Token-based 	Universal Time Series Model Multi-domain 
Ключевая особенность Уникальность	Масштабный pretraining  100M+ параметров	Вероятностный прогноз  CRPS optimization	Универсальность  Cross-domain
Zero-shot Способность	Отличная 90%	Отличная 95%	Хорошая 85%
Transfer Learning Перенос знаний	Fine-tuning  Adapter-based	Domain adaptation  Prompt-based	Cross-domain  Multi-task
Применение Сценарии использования	Финансы, IoT Finance IoT	Ритейл, энергетика Retail Energy	Универсальное All domains

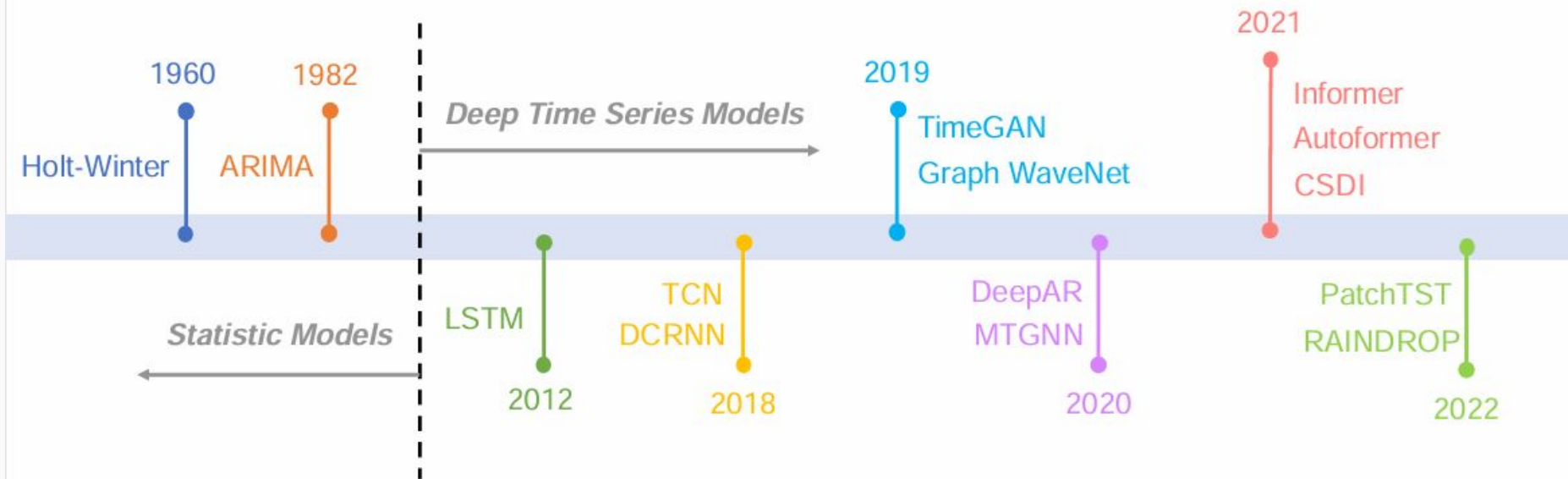
 **Рекомендация по выбору**

TimesFM — для крупных компаний с ресурсами на fine-tuning. Chronos — для вероятностного прогнозирования. Moirai — для универсальных сценариев.

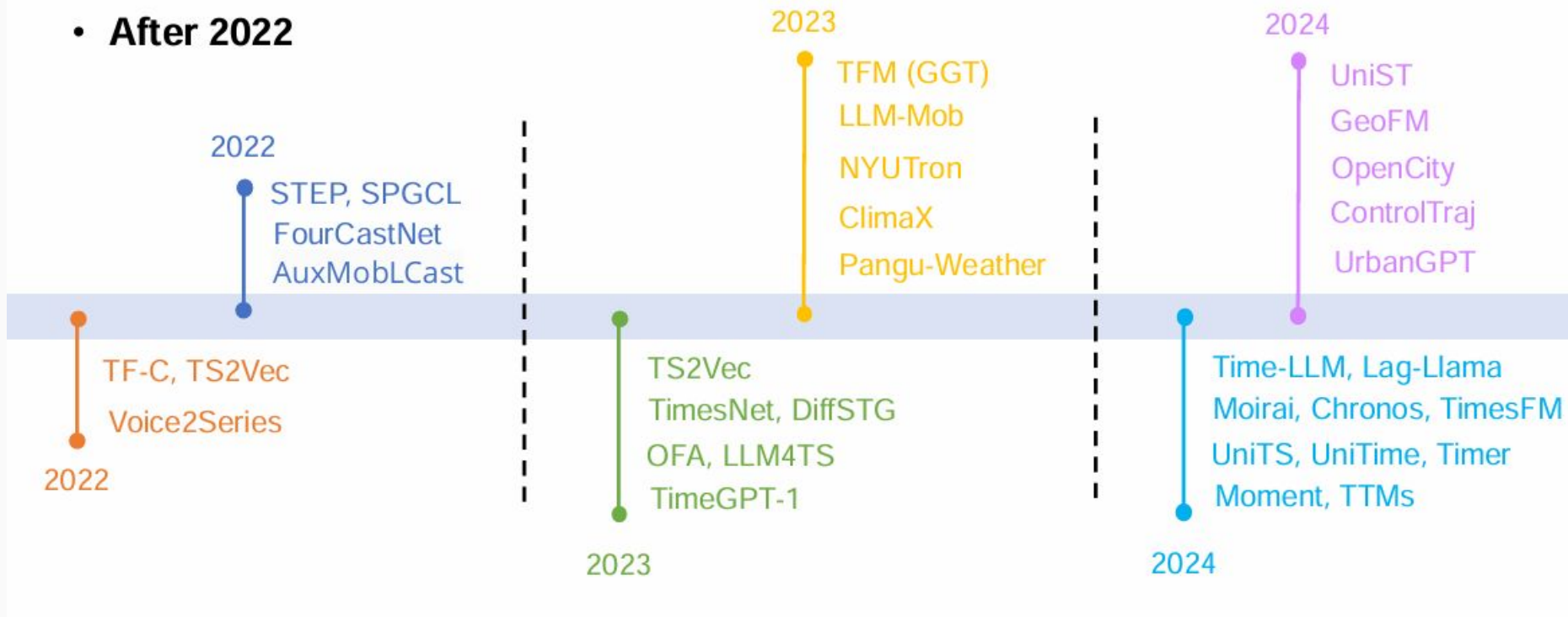


Timeline

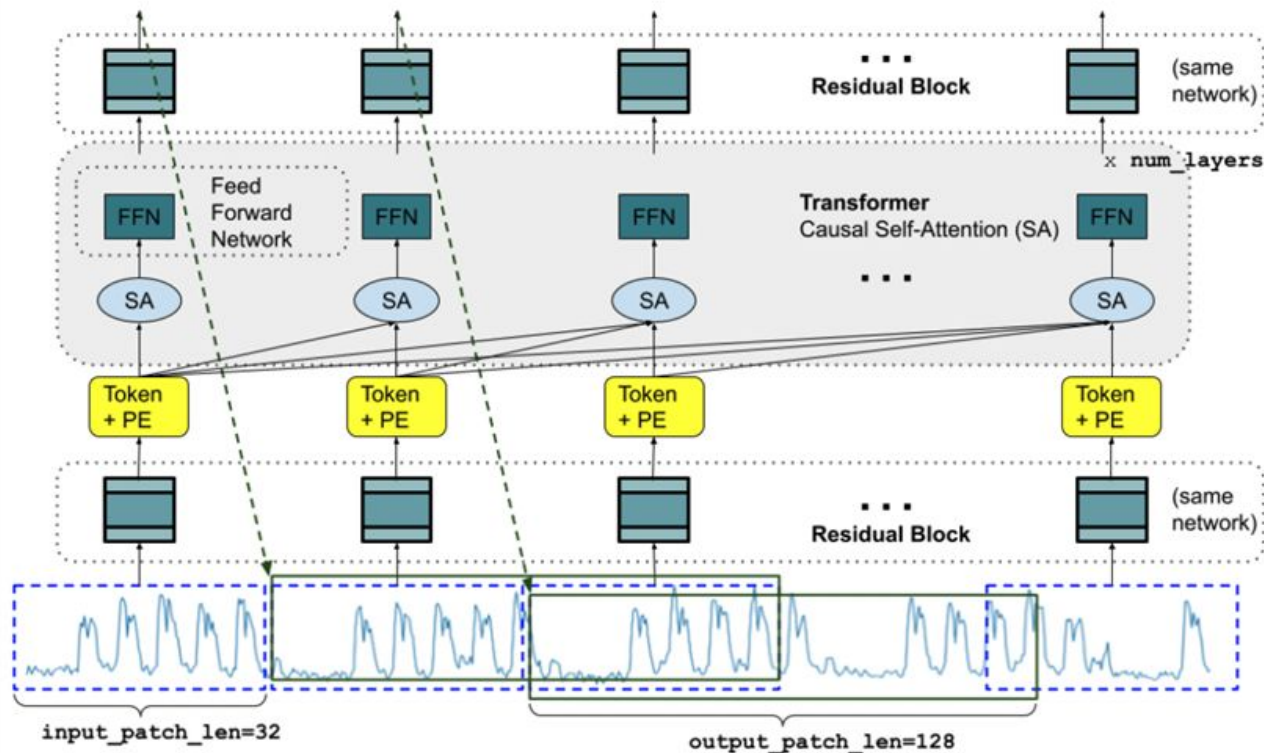
- Before 2022



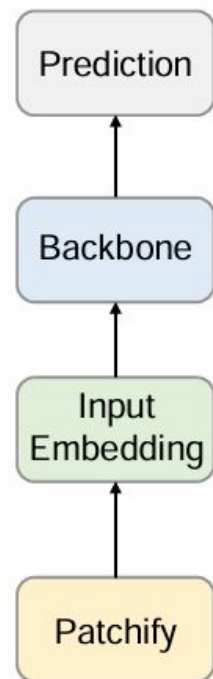
- **After 2022**



Transformer-based Models



(Decoder-only)



Zero-shot / Few-shot и перенос между доменами

Адаптация foundation models для временных рядов без дообучения или с минимальными данными



Zero-shot

Прямое применение

- ✓ **Прямой прогноз**
Без дообучения, используя промпт-контекст
- ✓ **Универсальность**
Работает на новых доменах
- ✓ **Экономия ресурсов**
Нет необходимости в обучении

📌 Идеально для быстрого прототипирования



Few-shot

Адаптация по примерам

- ✓ **Адаптация по примерам**
Несколько серий/окон для fine-tuning
- ✓ **Быстрая настройка**
Минимальные данные для адаптации
- ✓ **Лучшая точность**
Чем zero-shot для специфических задач

📌 Рекомендуется для специфических доменов



Transfer Learning

Перенос знаний

- ✓ **Fine-tune**
Адаптация модели под новый домен
- ✓ **Adapter layers**
Эффективная адаптация
- ✓ **Domain adaptation**
Сдвиг домена

📌 Эффективен при малых данных

Сравнение подходов

Метрики: MAE | RMSE

Zero-shot
0.15 MAE
Базовая точность

Few-shot (10 samples)
0.08 MAE
+47% улучшение

Fine-tune
0.05 MAE
+67% улучшение

Transfer Learning
0.06 MAE
+60% улучшение

Time Series Cross-Validation

Методы валидации для временных рядов: expanding vs rolling window



Expanding Window

Расширяющееся окно



1

Начальное обучение

Обучаем на первых n точках, валидируем на $n+1$

$t=1..n$

2

Расширение окна

Добавляем новую точку к обучению

$t=1..n+1$

3

Итерация

Повторяем до конца ряда

$t=1..n+k$



Преимущества

Использует все доступные данные, стабильная оценка



Rolling Window

Скольльзящее окно



1

Фиксированный размер

Окно постоянного размера сдвигается

$t=n..n+k$

2

Сдвиг окна

Сдвигаем окно на 1 шаг вперед

$t=n+1..n+k+1$

3

Итерация

Повторяем до конца ряда

$t=n+m..n+m+k$



Преимущества

Контроль размера окна, меньше вычислений